

ПРАКТИЧЕСКАЯ 8: КЛАСТЕРИЗАЦИЯ ДАННЫХ

Ожидаемое время выполнения: 3 часа (1,5 часа — выполнение кластеризации, 1,5 часа — анализ и оформление отчета).

Цель практики

Научиться проводить кластеризацию данных с использованием методов K-means и иерархической кластеризации в Python, R, а также визуализировать результаты кластерного анализа.

Задачи практики

1. Выполнить кластеризацию методом K-means:
 - Python: использование библиотеки `sklearn`.
 - R: функция `kmeans`.
2. Провести иерархическую кластеризацию:
 - Python: библиотека `scipy.cluster.hierarchy`.
 - R: функции `hclust`, `dendrogram`.
3. Проанализировать результаты кластеризации:
 - Интерпретация кластеров (центроиды, количество объектов в каждом кластере).
 - Сравнение кластеров, полученных разными методами.
4. Визуализировать результаты кластеризации:
 - Python: графики кластеров с использованием `matplotlib` и `seaborn`.
 - R: графическое представление дендрограмм и кластеров.
5. Сравнить удобство выполнения кластерного анализа в Python, R.

Формат отчета

1. Введение:
 - Цель и задачи работы.
 - Краткое описание методов кластеризации (K-means и иерархическая кластеризация).
2. Результаты работы:
 - Графики и результаты K-means:

- Python: коды и визуализация.
- R: результаты функции kmeans и визуализация.
- Графики и результаты иерархической кластеризации:
- Python: дендрограммы и распределение кластеров.
- R: визуализация с помощью hclust и дендрограмм.
- Таблица сравнения кластеров, созданных разными методами.

3. Заключение:

- Сравнение Python, R для проведения кластерного анализа.
- Рекомендации по выбору подходящего инструмента в зависимости от объема данных и задачи.

Дополнительные материалы к практической работе 8

Исходные данные: Набор данных с признаками для кластеризации (например, характеристики клиентов или продуктов).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 8

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Структура: Ваш набор данных должен содержать числовые признаки, пригодные для вычисления расстояний (например, метрик Евклида), или такие, которые можно легко закодировать/нормализовать.

Очистка:

Проверьте и обработайте пропущенные значения (импьютация либо исключение).

Удалите неинформативные признаки или те, которые напрямую не влияют на задачу кластеризации.

Масштабирование признаков (особенно для методов K-means):

Рекомендуется нормализовать или стандартизировать данные (MinMaxScaler, StandardScaler и т. д.), чтобы ни один из признаков не доминировал над остальными из-за масштабов.

1.2. Python (sklearn, scipy)

Импорт библиотек:

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Загрузка данных:

```
df = pd.read_csv('your_data.csv')  
print(df.head())
```

Проверка:

```
print(df.info())  
print(df.describe())
```

Масштабирование (рекомендуется для K-means и многих методов иерархической кластеризации):

```
scaler = StandardScaler()  
df_scaled = scaler.fit_transform(df)    # df - только числовые  
столбцы
```

1.3. R (kmeans, hclust)

Импорт и чтение данных:

```
# install.packages("ggplot2") при необходимости  
library(ggplot2)  
  
df <- read.csv("your_data.csv", header=TRUE)  
head(df)
```

Проверка структуры:

```
str(df)  
summary(df)
```

Очистка и масштабирование (если нужно):

```
# Предположим, у нас только числовые столбцы, которые мы сохраним  
в df_numeric  
df_numeric <- df[, sapply(df, is.numeric)]  
  
# Масштабирование  
df_scaled <- scale(df_numeric)
```

ШАГ 2. КЛАСТЕРИЗАЦИЯ МЕТОДОМ K-MEANS

2.1. Теоретические аспекты K-means

Цель: Разделить данные на k кластеров так, чтобы сумма квадратов расстояний от точек до центроидов (центров кластеров) была минимальна.

Начальное приближение: Изначально центроиды выбираются случайно (или посредством определённых алгоритмов инициализации).

Алгоритм:

Рассчитать расстояния от каждой точки до всех центроидов.

Присвоить точку ближайшему центроиду.

Пересчитать координаты центроидов как среднее по всем точкам кластера.

Повторять, пока не достигнута сходимость или не выполнено заданное число итераций.

2.2. Python (sklearn)

Запуск K-means

```
# Предположим, что df_scaled - это массив (numpy), полученный
# после масштабирования
k = 3 # количество кластеров, выберите исходя из задачи или
# эвристически
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(df_scaled)

# Предсказанные кластеры
labels = kmeans.labels_

# Центроиды
centroids = kmeans.cluster_centers_
```

Анализ результатов

```
print("Кластеры для объектов:\n", labels)
print("Координаты центроидов:\n", centroids)
```

labels содержит номер кластера для каждой точки.

centroids содержит координаты центроидов (в масштабе масштабированных данных).

Визуализация (пример для 2D или 3D)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)

plt.scatter(df_pca[:,0], df_pca[:,1], c=labels)
plt.title("K-Means (2 компоненты PCA)")
plt.show()
```

2.3. R (функция kmeans)

Запуск K-means

```
set.seed(42) # для воспроизводимости
k <- 3       # количество кластеров
kmeans_result <- kmeans(df_scaled, centers = k, nstart = 10)
# nstart - количество случайных инициализаций
```

Анализ результатов

```
kmeans_result$cluster # вектор принадлежности кластеру
kmeans_result$centers # координаты центроидов
kmeans_result$tot.withinss # суммарная внутрикластерная сумма квадратов
```

Визуализация (Аналогично Python, визуализировать в пространстве PCA):

```
library(ggplot2)

pca_res <- prcomp(df_scaled)
df_pca <- data.frame(pca_res$x[,1:2], Cluster =
factor(kmeans_result$cluster))

ggplot(df_pca, aes(x=PC1, y=PC2, color=Cluster)) +
  geom_point() +
  ggtitle("K-means Clustering (PCA projection)")
```

ШАГ 3. ИЕРАРХИЧЕСКАЯ КЛАСТЕРИЗАЦИЯ

3.1. Теоретические аспекты

Агломеративный подход: начинается с предположения, что каждый объект сам по себе кластер, далее кластеры последовательно объединяются по определённому критерию (linkage — средний, полный, одиночный).

Результат: дендрограмма, на которой можно «обрезать» дерево на нужном уровне, чтобы получить заданное количество кластеров.

3.2. Python (scipy.cluster.hierarchy)

Построение дендрограммы

```
import scipy.cluster.hierarchy as sch

# linkage: метод объединения, например "ward" (минимизация роста
# внутрикластерной суммы квадратов)
Z = sch.linkage(df_scaled, method='ward')

# Построение дендрограммы
plt.figure(figsize=(10, 7))
dn = sch.dendrogram(Z)
plt.title("Иерархическая кластеризация (дендрограмма)")
plt.show()
```

Выделение кластеров

```
# Допустим, нужно выделить 3 кластера
clusters = fcluster(Z, t=3, criterion='maxclust')
print(clusters)
```

3.3. R (hclust, dendrogram)

Построение дерева

```
dist_matrix <- dist(df_scaled, method="euclidean") # матрица
расстояний
hc <- hclust(dist_matrix, method="ward.D2") # ward.D2
```

часто используется

```
# Построение дендрограммы  
plot(hc, main="Иерархическая кластеризация", xlab="", sub="")
```

Выделение кластеров

```
# Допустим, нужно 3 кластера  
clusters <- cutree(hc, k=3)  
table(clusters)
```

ШАГ 4. АНАЛИЗ РЕЗУЛЬТАТОВ КЛАСТЕРИЗАЦИИ

Сравните распределение объектов по кластерам при K-means и иерархической кластеризации:

Сколько объектов в каждом кластере.

Основные характеристики кластеров (центроиды, средние значения признаков).

Интерпретация:

Каковы общие черты объектов в каждом кластере? (Например, «Кластер 1 содержит клиентов с высокими затратами и низкой посещаемостью».)

Совпадают ли выделенные кластеры с предположениями о структуре данных?

Оцените стабильность кластеров, возможные аномалии, пересечения между методами (иногда объекты распределяются по-разному из-за разницы алгоритмов).

ШАГ 5. ВИЗУАЛИЗАЦИЯ КЛАСТЕРОВ И СРАВНЕНИЕ В ИНСТРУМЕНТАХ

5.1. Python

Использовать PCA-проекцию для 2D/3D-графиков (пример кода уже приведён выше).

Можно добавлять дополнительные признаки в парные диаграммы (pairplot) с окрашиванием по кластерам.

5.2. R

Аналогично использовать prcomp для PCA и визуализации 2D/3D.

Пример уже представлен, можно расширять под ваши нужды (добавить facet_wrap, цветовые шкалы и т.д.).

КЛЮЧЕВЫЕ МОМЕНТЫ ДЛЯ УСПЕШНОЙ ЗАЩИТЫ ПРАКТИКИ

Масштабирование данных: продемонстрируйте, что вы понимаете его важность, особенно для K-means.

Выбор числа кластеров: хотя в инструкции напрямую не говорится об оптимизации k, полезно кратко упомянуть методы вроде «локтя» (Elbow method) или «силуэтного анализа» (Silhouette Score).

Интерпретация: не просто покажите кластеры, но и расскажите, что значит каждый кластер (какие признаки в нём характерны).

Сравнение: подчеркните, чем результаты K-means отличаются от

иерархической кластеризации, и какие выводы можно сделать.

Такое пошаговое руководство позволит выполнить кластеризацию в двух инструментах, визуализировать результаты и сравнить их, а также даст возможность сформировать структурированный отчёт для завершения практической работы по кластерному анализу.